

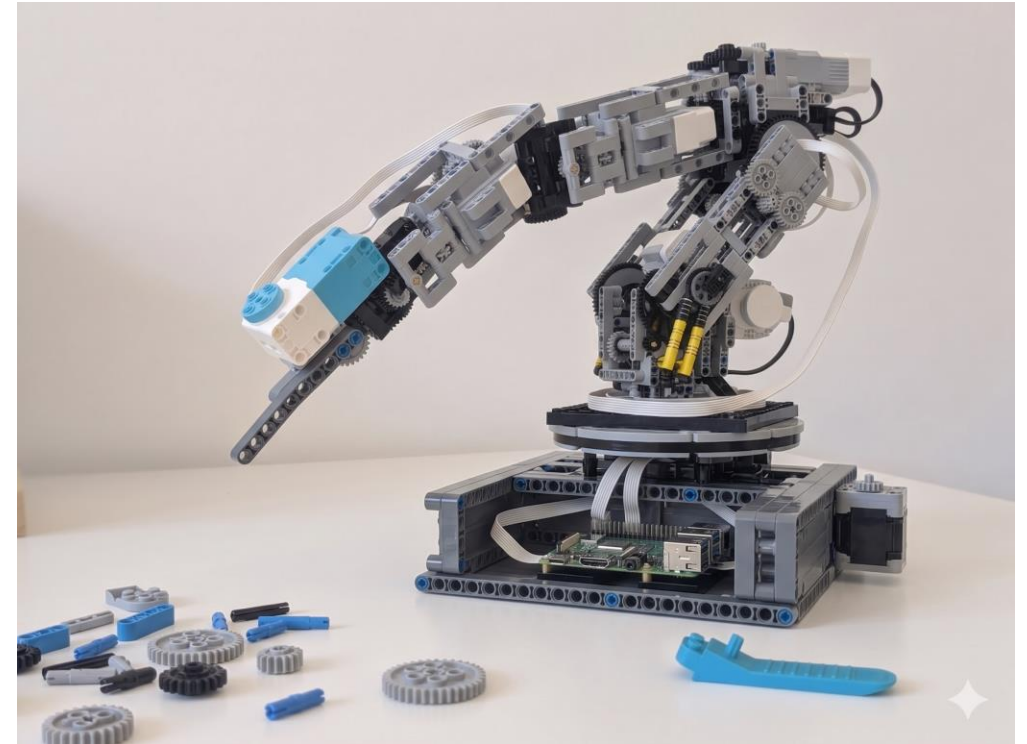
Second Milestone (Team 1)

Waleed Khaled

Nils Rheinländer

◆ What is the Project about?

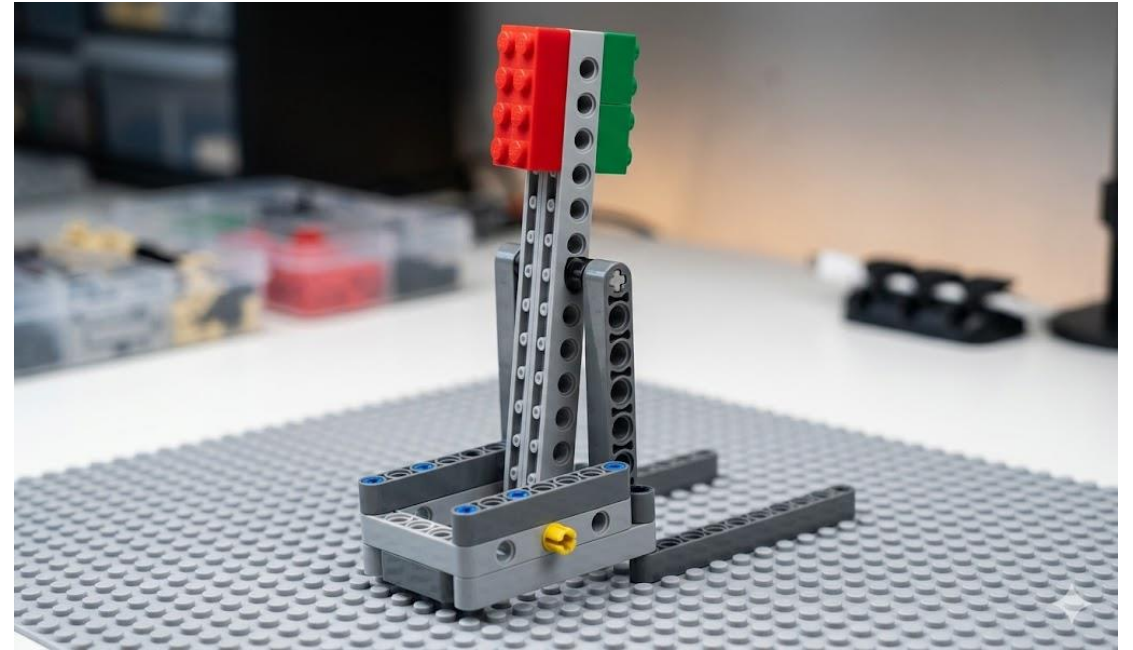
- Build a custom LEGO robot arm
- Perform a specific manipulation task
- No handcrafted algorithms
- Use a Vision-Language-Action model



Prototype: Robot (Nano banana)

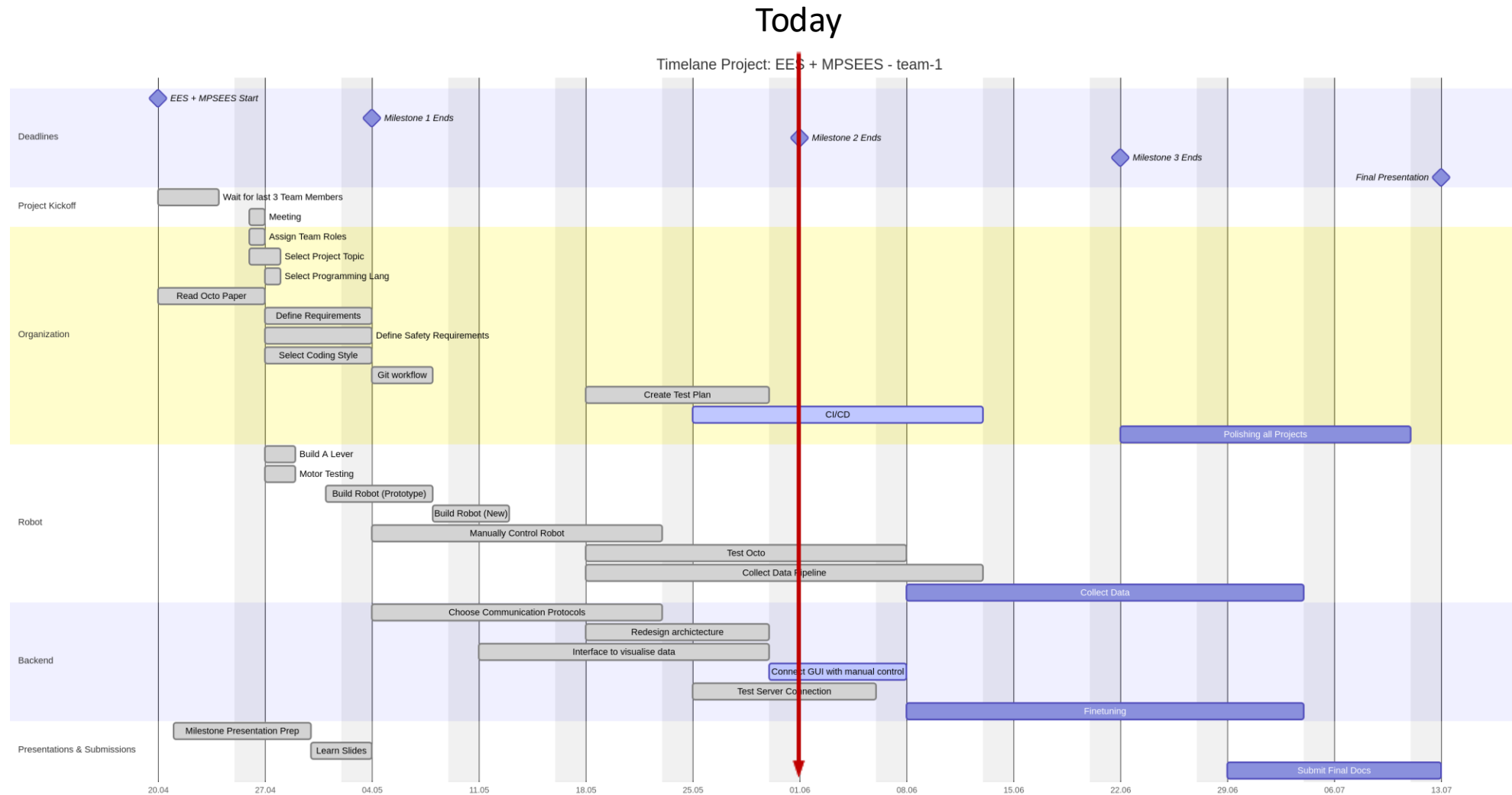
◆ Project Goal

- Train a LEGO arm to flip a marked lever using a finetuned Octo model

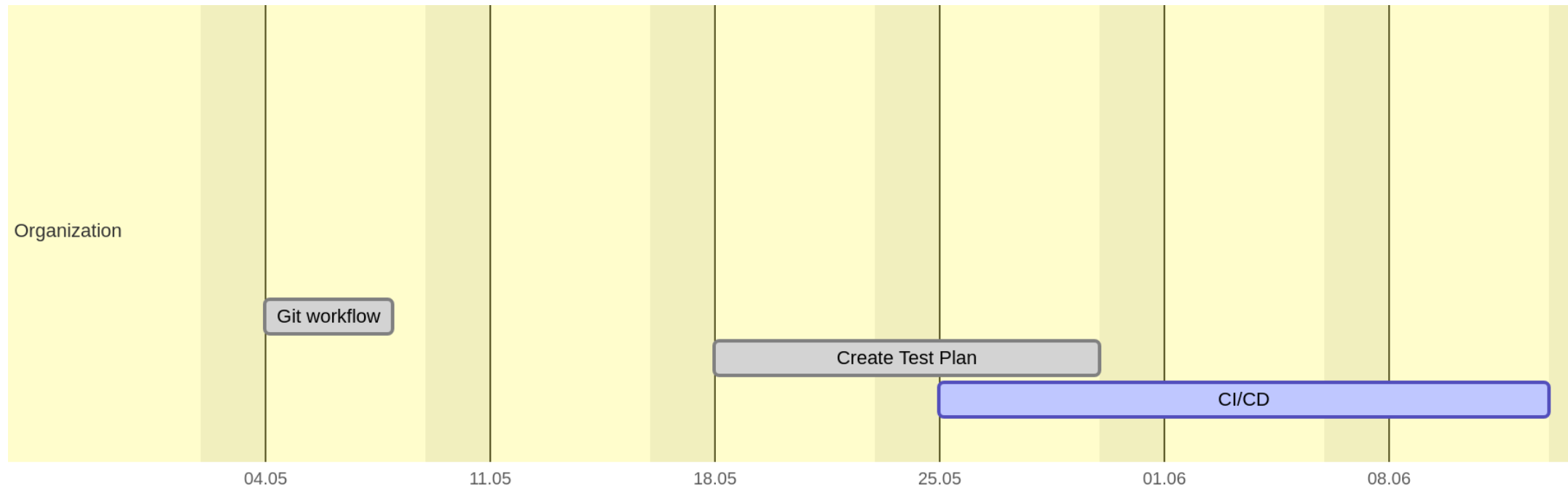


Prototype: lever with red/green sides

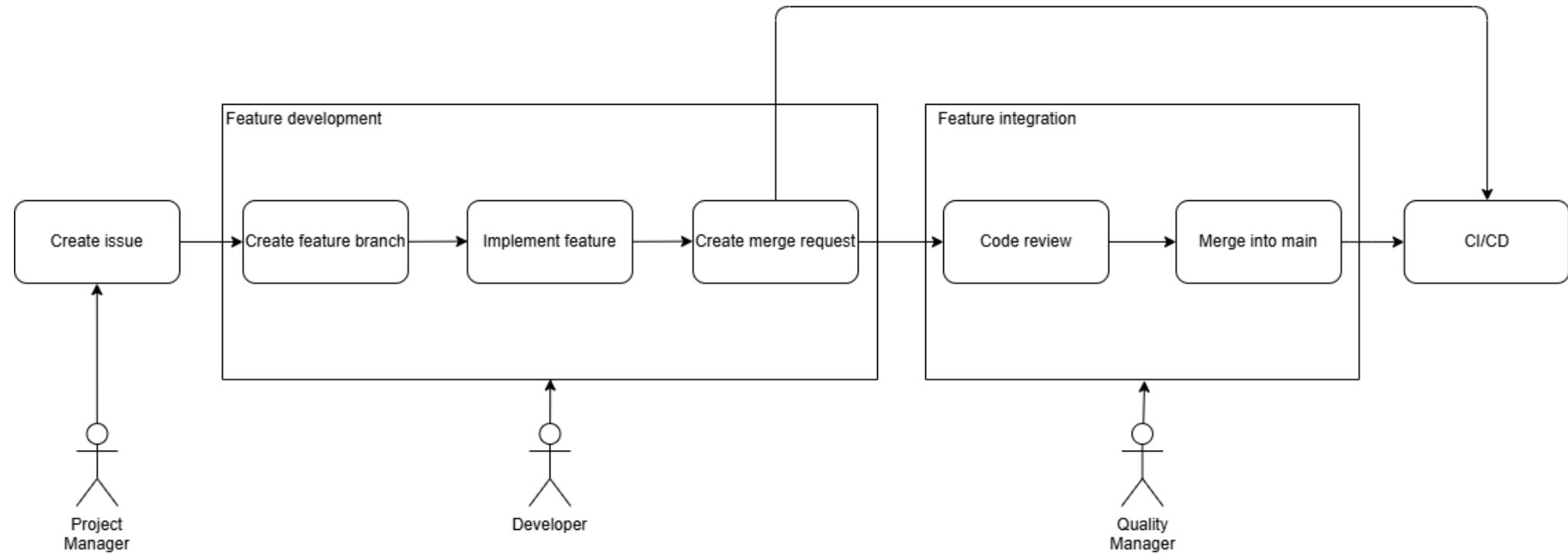
GANTT Chart



◆ GANTT Chart (Milestone 2)



Git Workflow



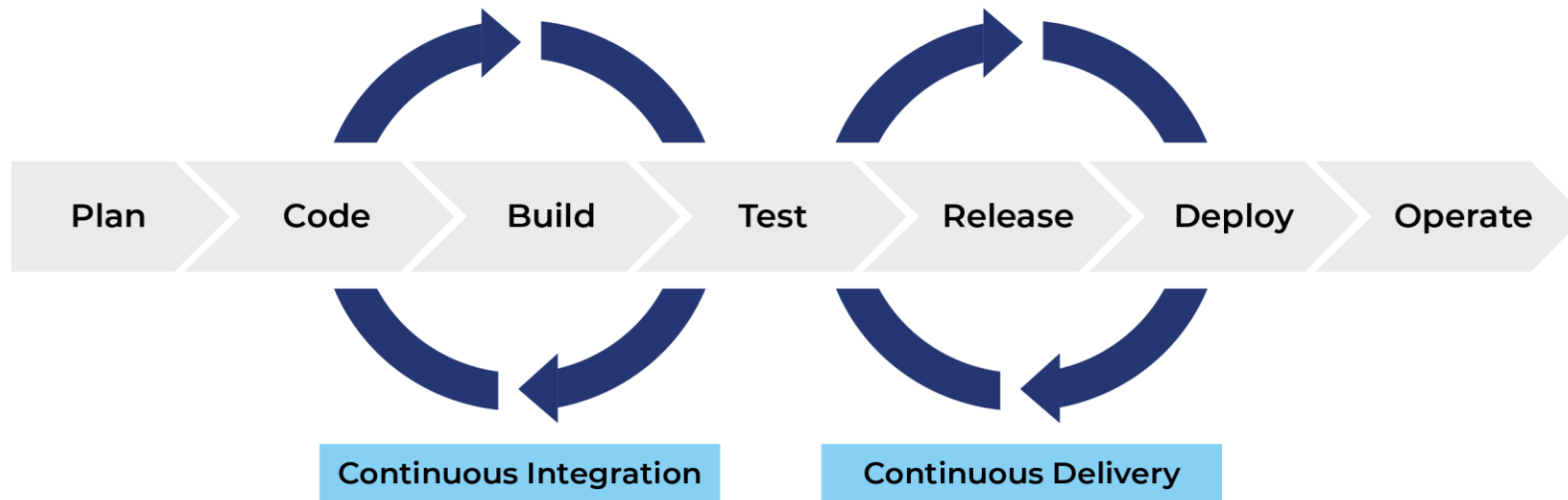
◆ CI/CD Pipeline

■ Structure

- Automatic CI runner for tests and linting
- Two separate pipelines

■ Backend vs. Robot

- Docker container validation
- CD produces deployable artifacts
- Reproducible code



[1] CI/CD Pipeline

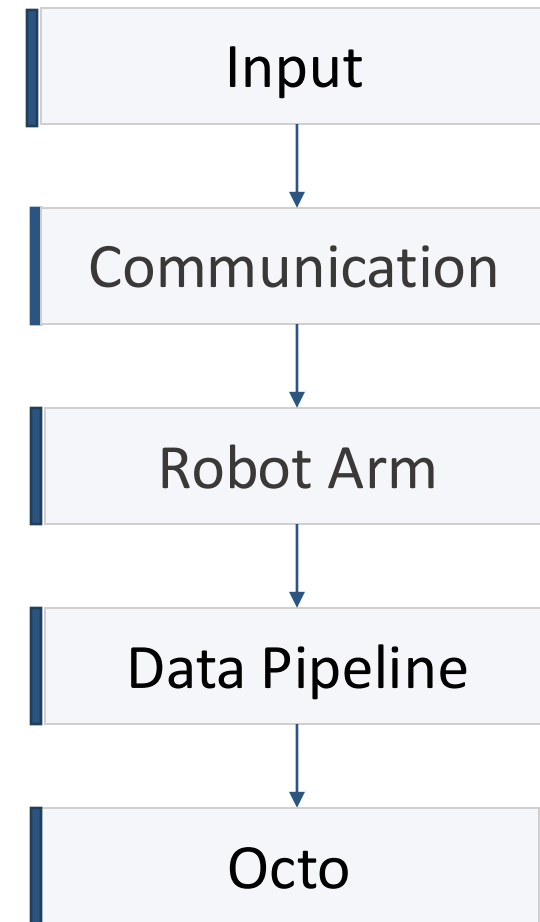
◆ Testplan - Strategy

■ Main Objectives

- Reliable teleoperation
- Stable communication
- Correct trajectory recording
- High-quality training data

■ Systems to be tested

- Robot arm & BuildHAT
- Pi server
- Controller & Networking
- Recording & Inference pipeline



End-to-End Validation Pipeline

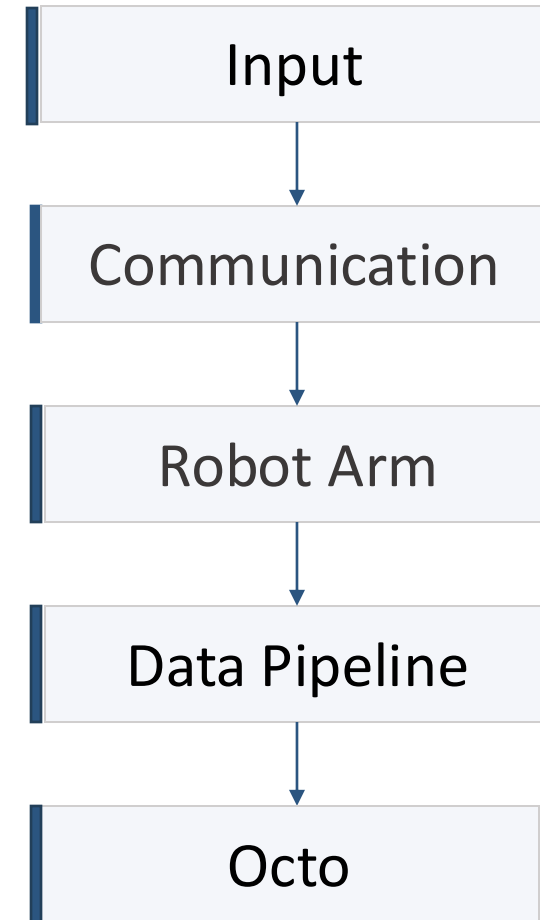
◆ Testplan - Manual & Automated Tests

■ Manual Tests

- Encoder consistency
- Motor positioning
- Teleoperation responsiveness
- Repeatability of trajectories

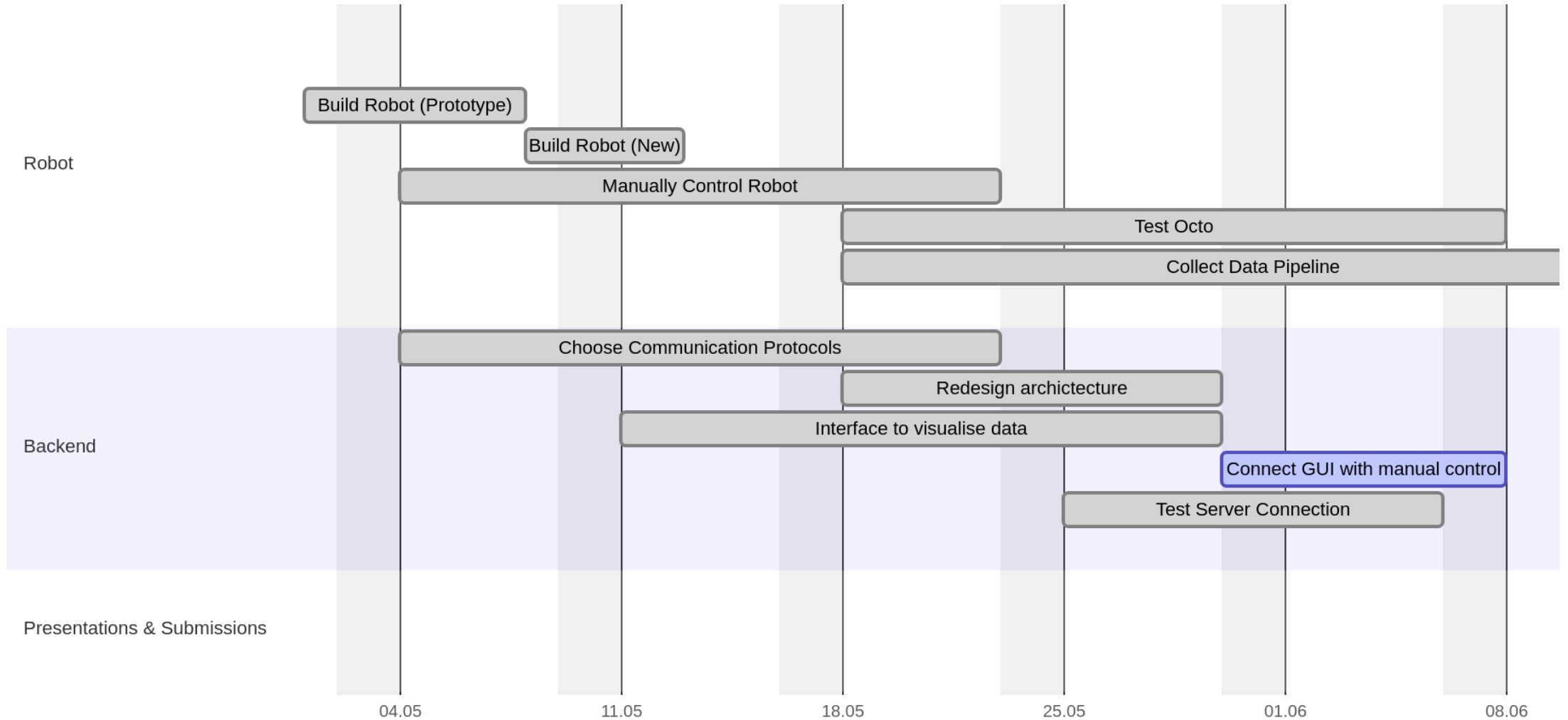
■ Automated Tests

- Kinematics
- Synchronization
- Unit tests

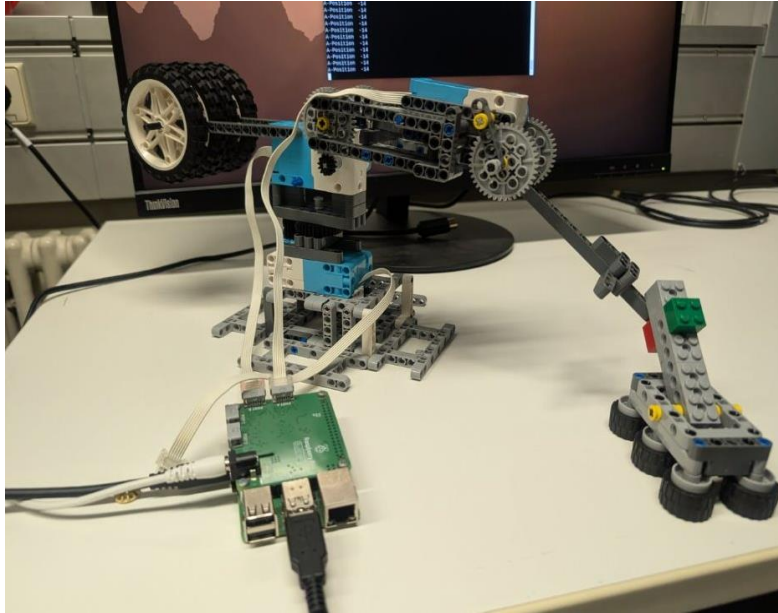


End-to-End Validation Pipeline

◆ GANTT Chart (Milestone 2)



◆ New Robot Design



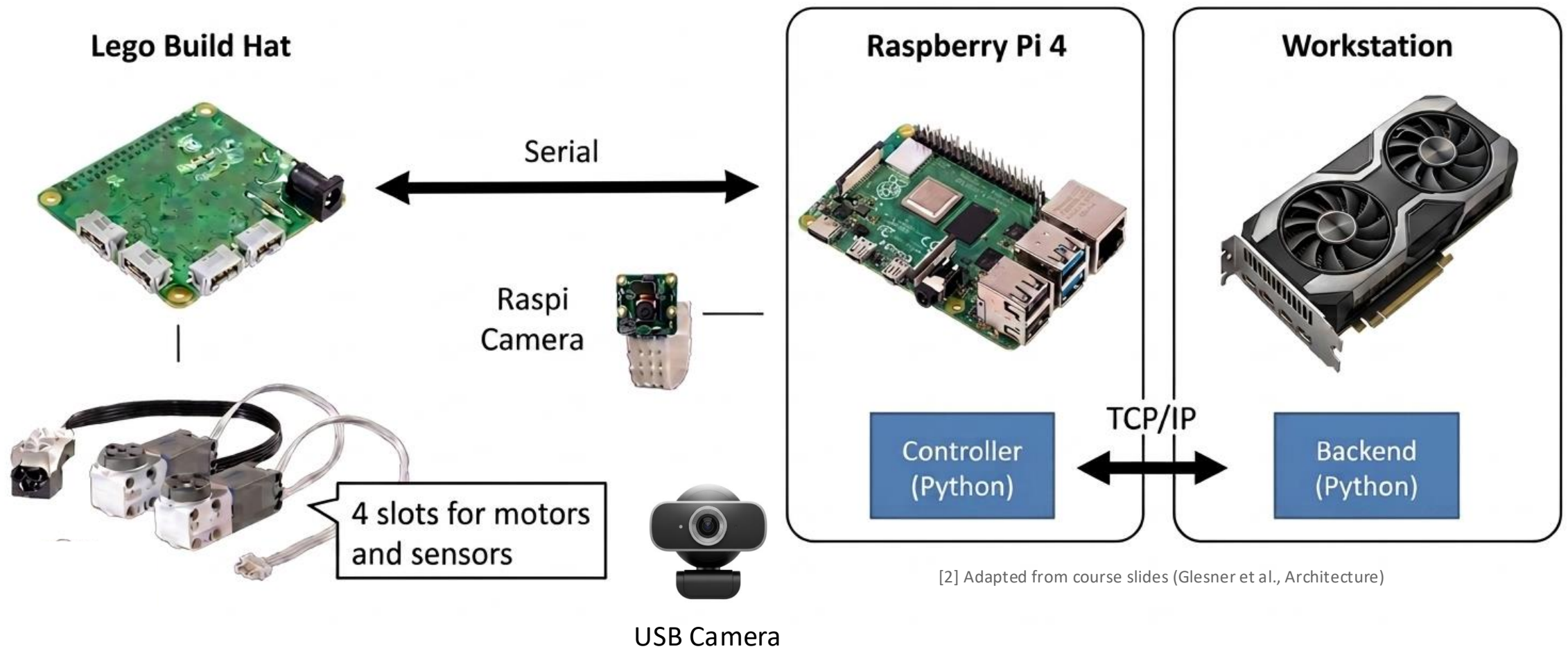
Switcher v1.0



Switcher v1.1

- Optimized balance
- Resolved motor cable length issue
- Mounted camera

◆ System Architecture



◆ Project Phases

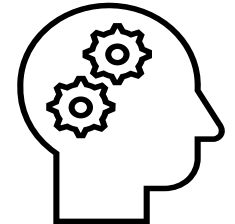
Phase 1: Data Collection

- Pi: Streams telemetry & executes manual commands.
- Laptop: Maps Xbox inputs & logs paired state-action data locally.
- Octo Model: Inactive.



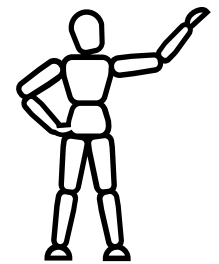
Phase 2: Offline Finetuning

- Pi: Disconnected
- GPU Workstation: Compiles the dataset & trains the model weights.
- Octo Model: Learning.



Phase 3: Autonomous Inference

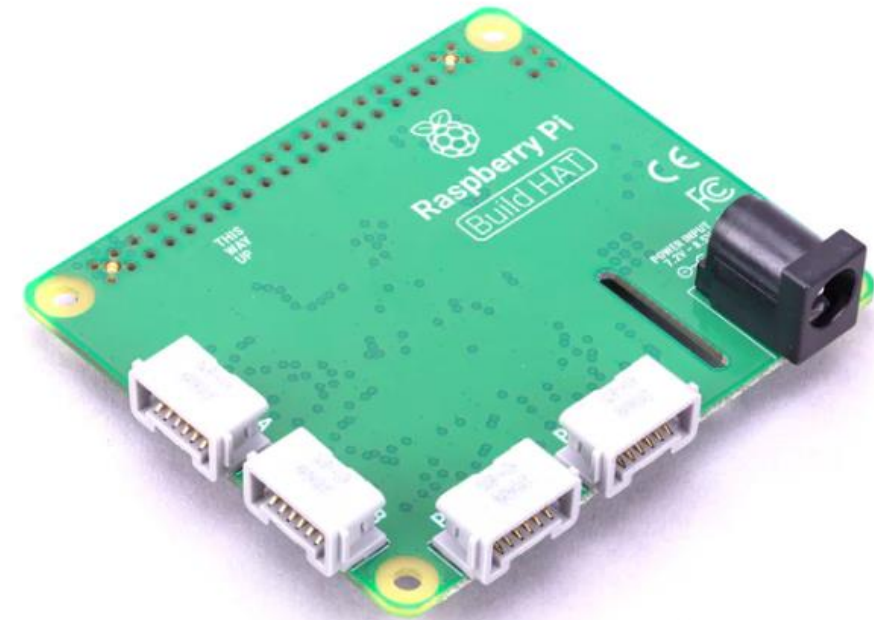
- Pi: Streams live state & executes AI-generated commands.
- GPU Workstation: Runs Octo inference & calculates Inverse Kinematics (IK).
- Octo Model: Active (Driving).



◆ Client-Side: Edge Environment

Raspberry Pi 4 / Edge Node

	Build HAT Manager
	3x LEGO Spike Technic Motors
	Raspberry Pi / USB Camera (Telemetry)
	Hardware Limit Enforcer
	TCP Client Interface



[3] Raspberry Pi Build Hat

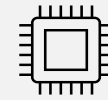
Responsible for real-time I/O, hardware safety, and telemetry serialization.

◆ Server-Side: Compute Environment



[4] Example GPU Workstation architecture

GPU Workstation / Server Node



Octo Foundation Model (JAX/Flax)



Pinocchio IK Solver (Task $\rightarrow$ Joint)



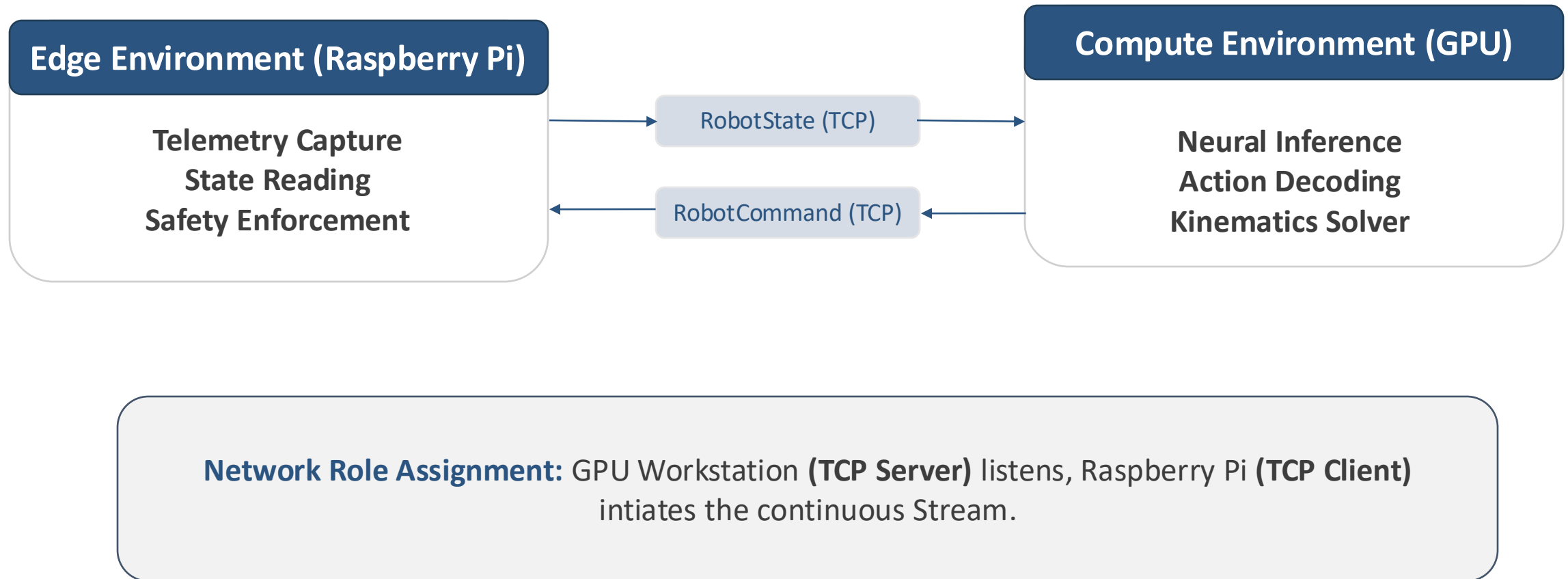
Octo Wrapper (Tokenization)



TCP Server (Socket srv.py)

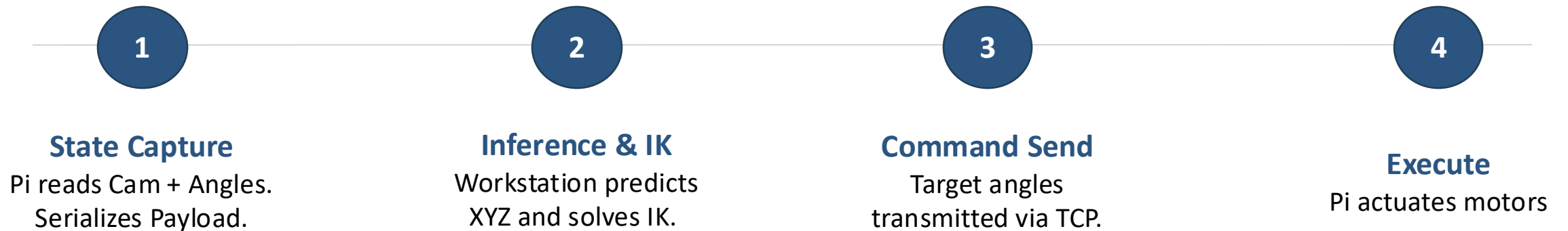
Handles computationally intensive neural inference and inverse kinematics.

Unified System Architecture



◆ Inference Execution Loop

Synchronous 5 Hz (200ms) Frame



Strict Latency Constraint: Network transmission, inverse kinematics, and neural inference must collectively execute in under 200ms to ensure fluid physical actuation.

Latency Fault Handling: Cycles exceeding 200ms are explicitly flagged as errors, and stale command packets are dropped

◆ Technical Stack



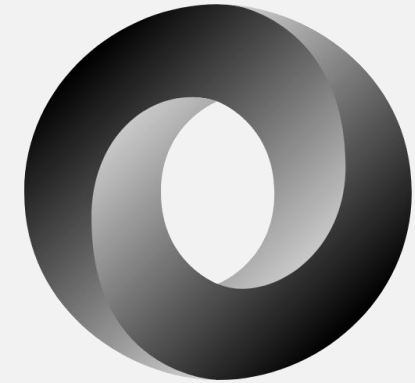
OpenCV

High-speed JPEG encoding (imencode) for real-time telemetry streaming



Python Pickle

Serializes Python objects into byte streams for low-latency, real-time TCP transmission of states and commands.



JSON

Human-readable metadata Storage for initial goals, and episode evaluation metrics

◆ Zero Calibration Paradigms

Analysis of rejected Homing Paradigms

Approach	Critical Flaw & Constraint
Relative Tracking	Encoder memory is volatile, power lost results in lost origin.
Limit Switches	Fitting Force Sensors adds wiring hazards and unacceptable mechanical complexity.
Hard-Stop Stall	Plastic gear sets are susceptible to fatigue and stripping, base joint lacks physical stops.



[5] LEGO Spike Force Sensor

◆ Zero Calibration – Chosen Approach

Manual Calibration:

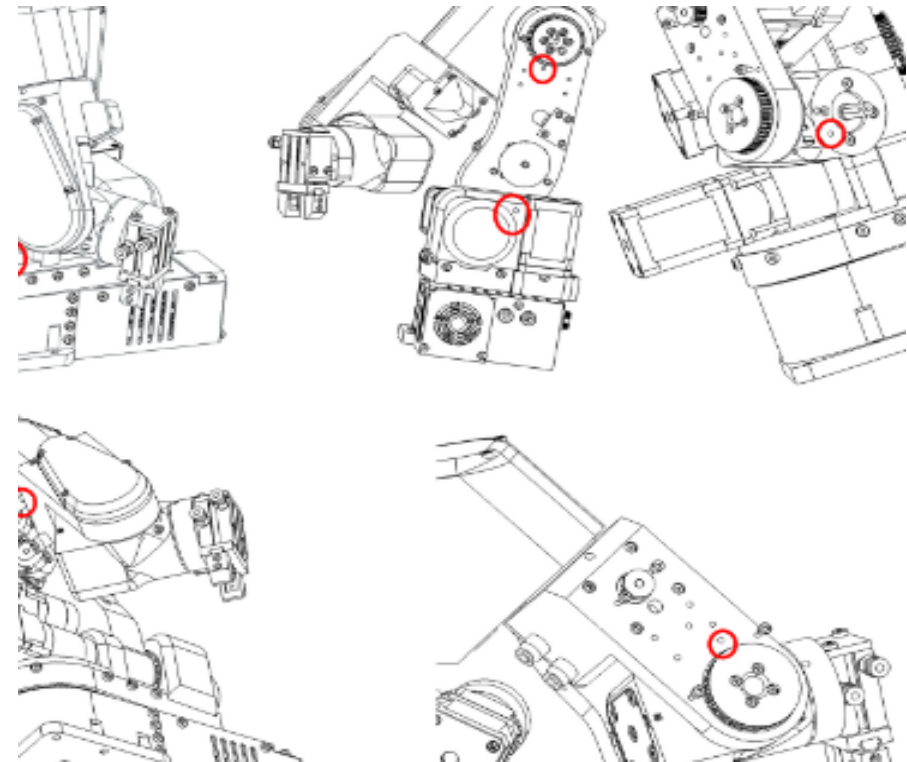
We combine manual alignment with software-enforced limits.

- **Phase 1: Witness Marks**

Zero-stress manual alignment. Octo's visual feedback naturally compensates for any mm-scale variance.

- **Phase 2: One-Time Limit Sweep**

An empirical sweep discovers true travel boundaries relative to zero. Values are permanently encoded in the URDF.



[6] Industrial robotic calibration schematics (Witness Marks).

◆ Zero Calibration – Software Features

Post-Session Null Reset

- Upon session termination, software instructs all joints to return to the null position (0,0,0).
- This eliminates manual physical resets between consecutive runs.

Persistent State Tracking

- Revolution count(s) are cached in a file. The system resumes by loading previous states.
- This reduces the frequency of required manual calibration.

Critical Risk: Zero-Point Drift

Drastic movement while powered off causes origin loss, requiring manual Witness Mark recalibration.



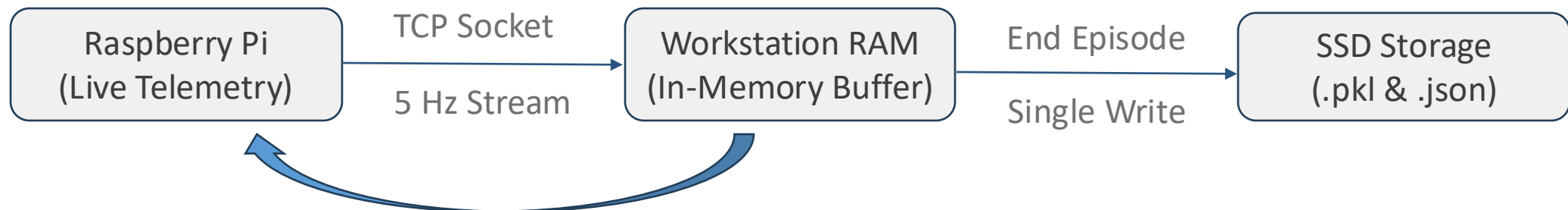
◆ Real Time Data Logging

1. Micro-Level (Step Buffering):

- Telemetry: Pi transmits RobotState via TCP.
- RAM Allocation: Workstation pairs state/command and appends to an internal memory list.

2. Macro-Level (Episode):

- Pickle Payload: At end_episode, the RAM buffer is serialized into a single .pkl file.
- JSON Metadata: Stores initial goal, initial position, end position, and success flag.

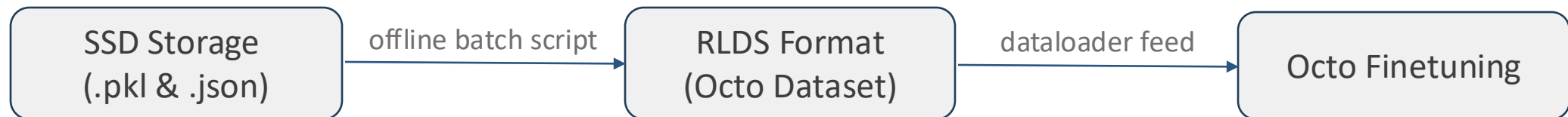


Offline Finetuning Preparation

While Pickle is efficient for memory dumping, it introduces severe parsing latency during GPU finetuning. We utilize an offline two-step pipeline.

RLDS Conversion Script:

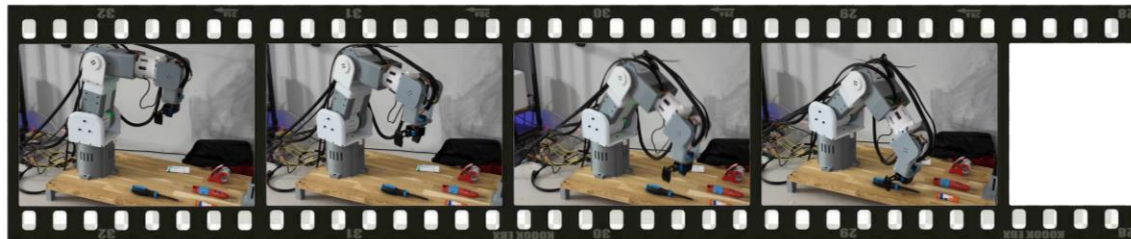
- **Aggregation:** Iterates through all isolated episode.pkl files.
- **Tensor Formatting:** Decodes JPEGs and aligns arrays to Octo's expected dimensions.
- **TFRecord:** Outputs a unified, chunked binary dataset.



◆ Octo Test

■ Octo Notebook

- Loaded pretrained Octo model
- Single-step action inference (e.g. pick up the fork)
- Trajectory-level action inference (e.g. pick up the fork based on history)

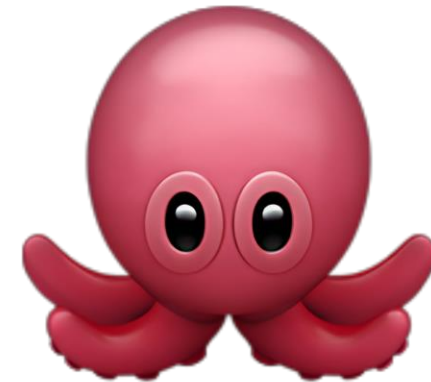


[8] Time series prediction

■ Further steps

- Test Octo base model on our robot

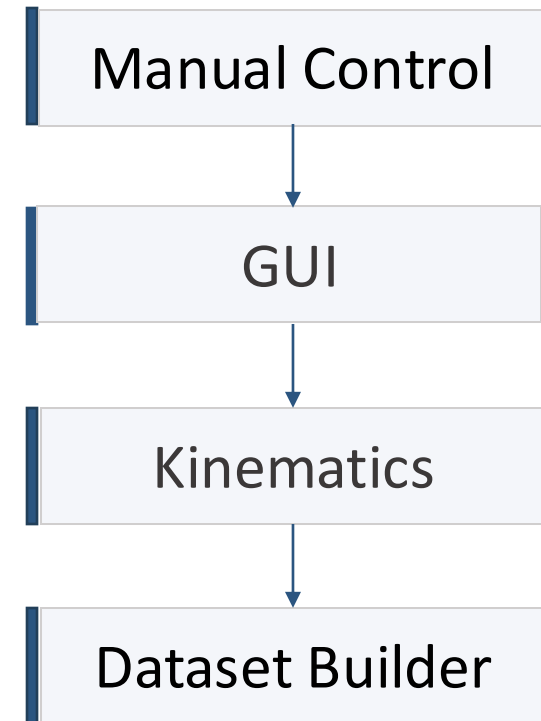
colab



[7] Octo

◆ Data Collection - Goal

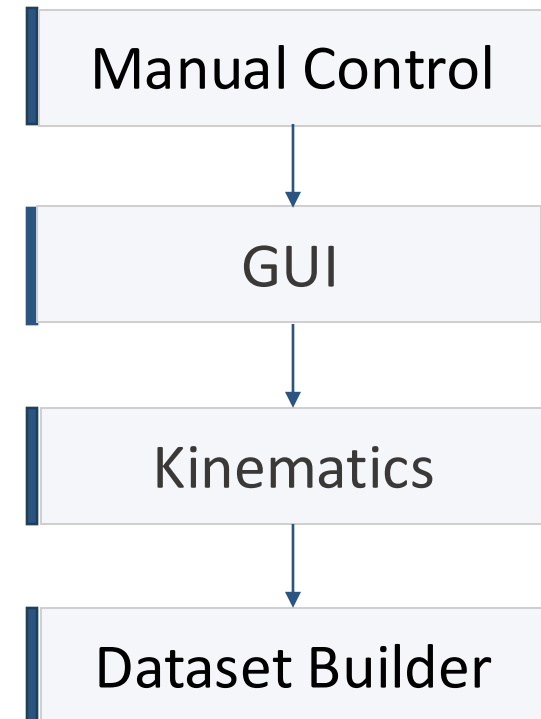
- **Goal:** Create a dataset to finetune the Octo model.



◆ Data Collection - Approach

■ Method

- Teleoperation via X-Box controller
- Monitoring and recording via GUI
- Calculate endeffector positions
- Generate a dataset using multiple examples (e.g. language instructions, positions, ...)



◆ Data Collection – Manual Robot Control

- Live joystick teleoperation of the 3-DOF
- Robust control (deadzones, ...)
- Logging data (motor positions, ...)
- Dual camera recording



[9] X-Box Controller

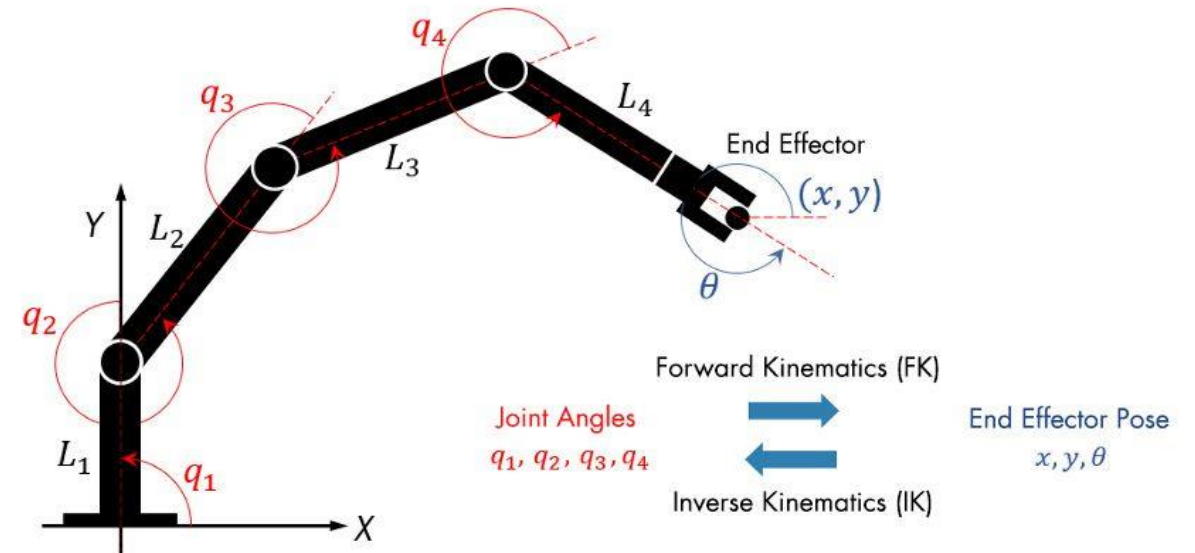
◆ Data Collection – Kinematics

■ Kinematics types:

- Forward kinematics
- Inverse kinematics

■ Python library: pinocchio

- Open source
- Focusing on robotic



[10] Mathworks: Kinematics



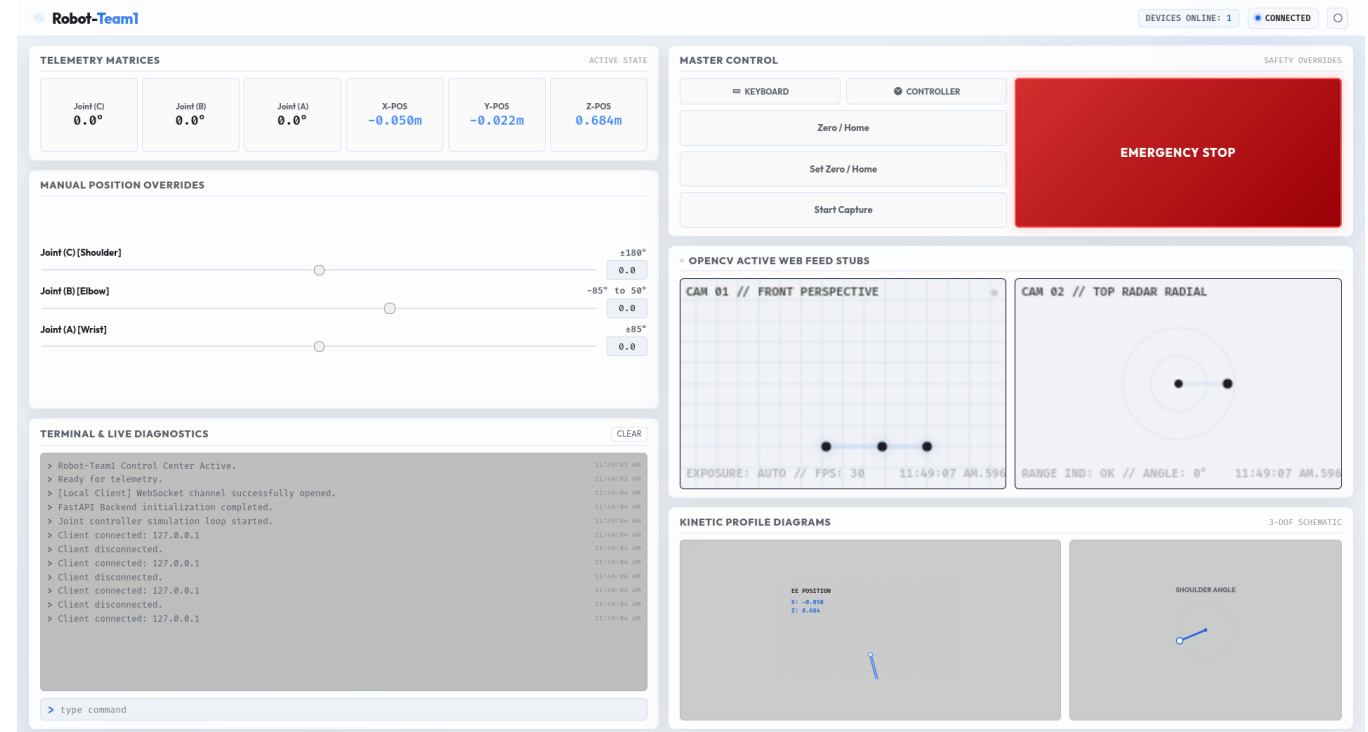
◆ Data Collection – GUI

■ Python Backend:

- FastAPI
- Websocket
- Uvicorn

■ Typescript Frontend:

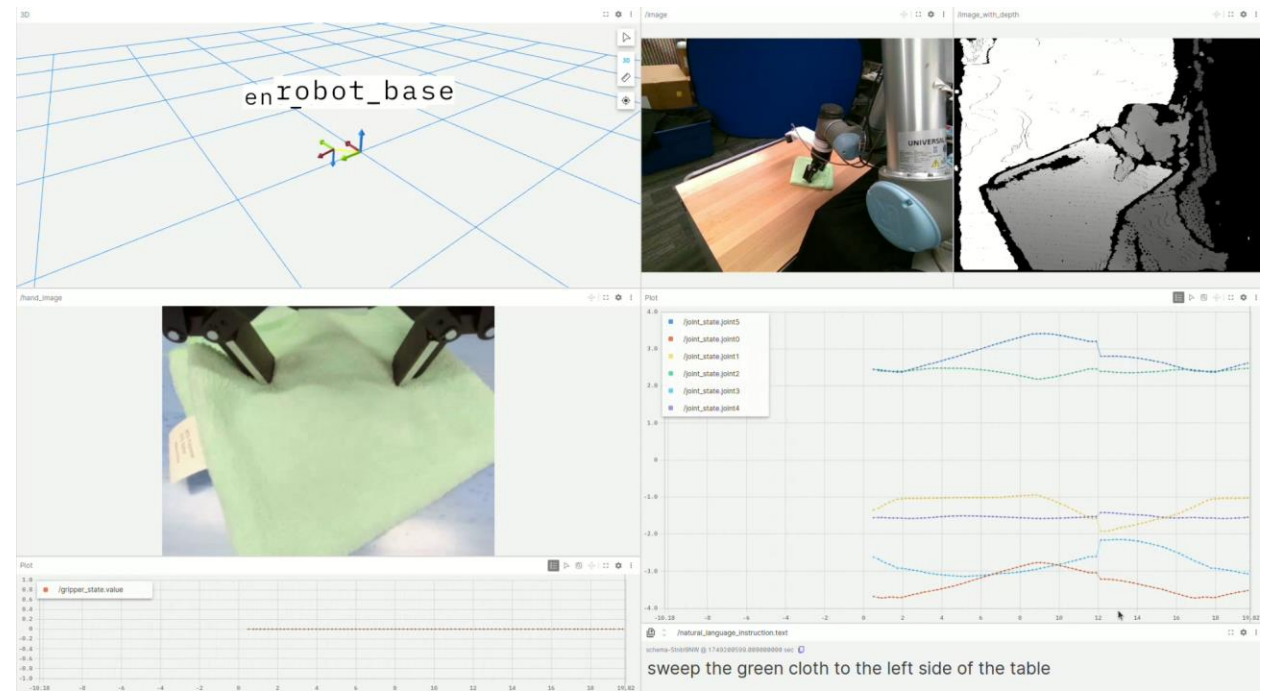
- Sveltekit for CSR
- Tailwindcss for styling



Gui Demo

◆ Data Collection – RLDS

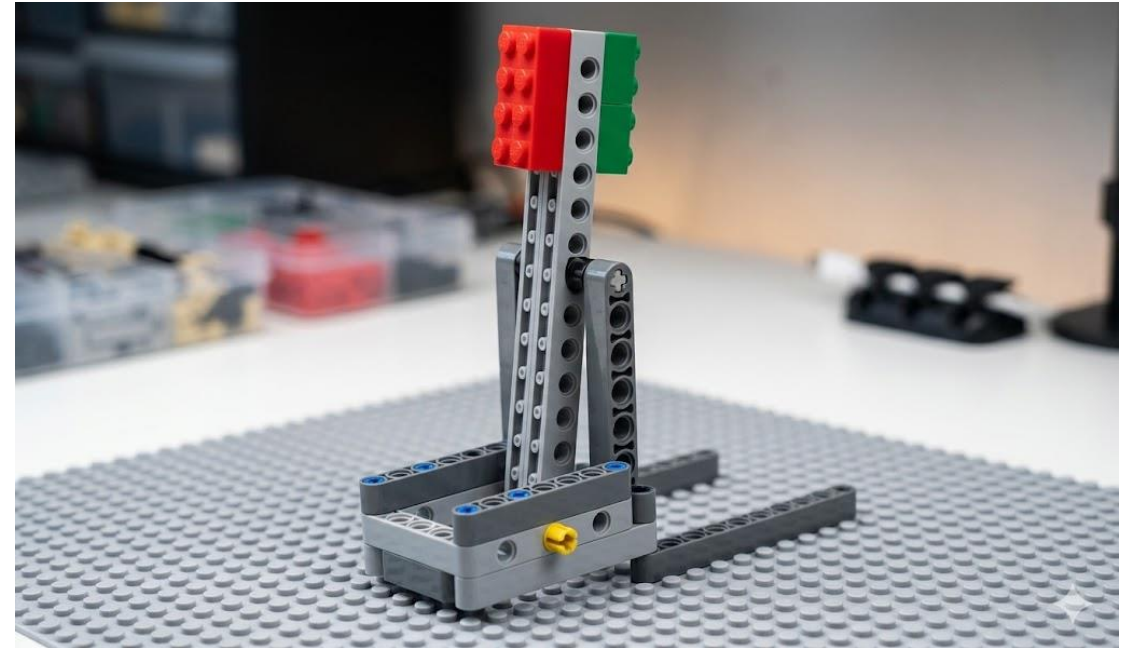
- RLDS dataset builder from the X-embodiment experiment[11]
- Record several parameters:
 - Camera
 - Wrist camera
 - Language instruction
 - Motor positions
 - ...



[12] Foxglove, 2025, Fig. 1

◆ Data Collection - Quality

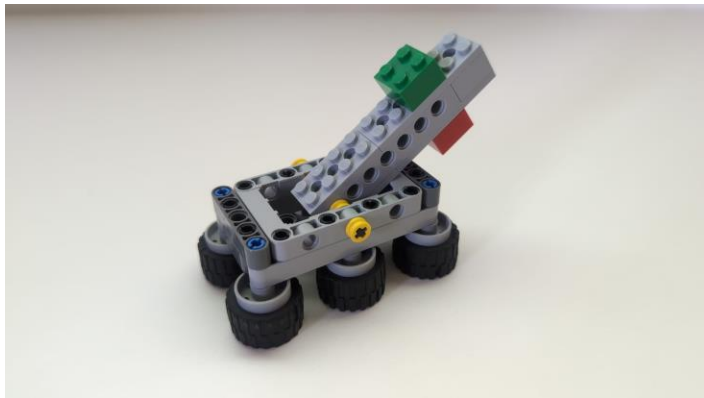
- Modify key variables:
 - Lever positions
 - Robot positions
 - Language instructions



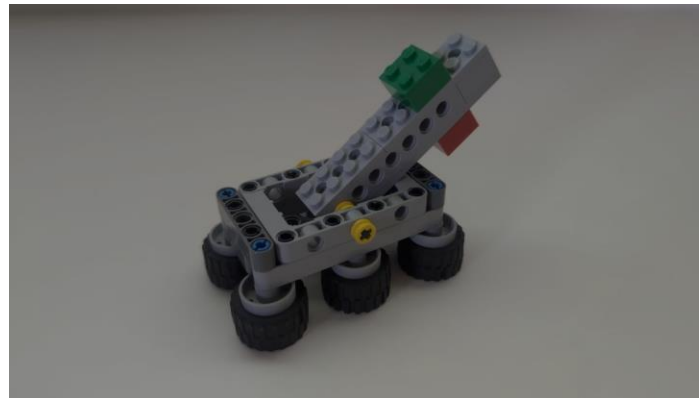
Prototype: lever with red/green sides

◆ Data Collection - Augmentation

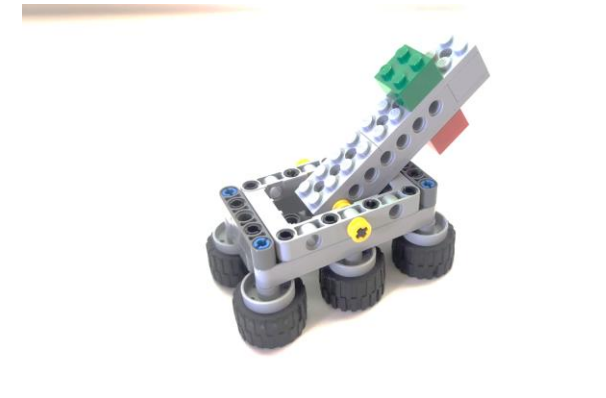
- Brightness variation, camera randomization every couple of demos



Original frame captured during teleoperation



Darkened to handle bright lighting conditions

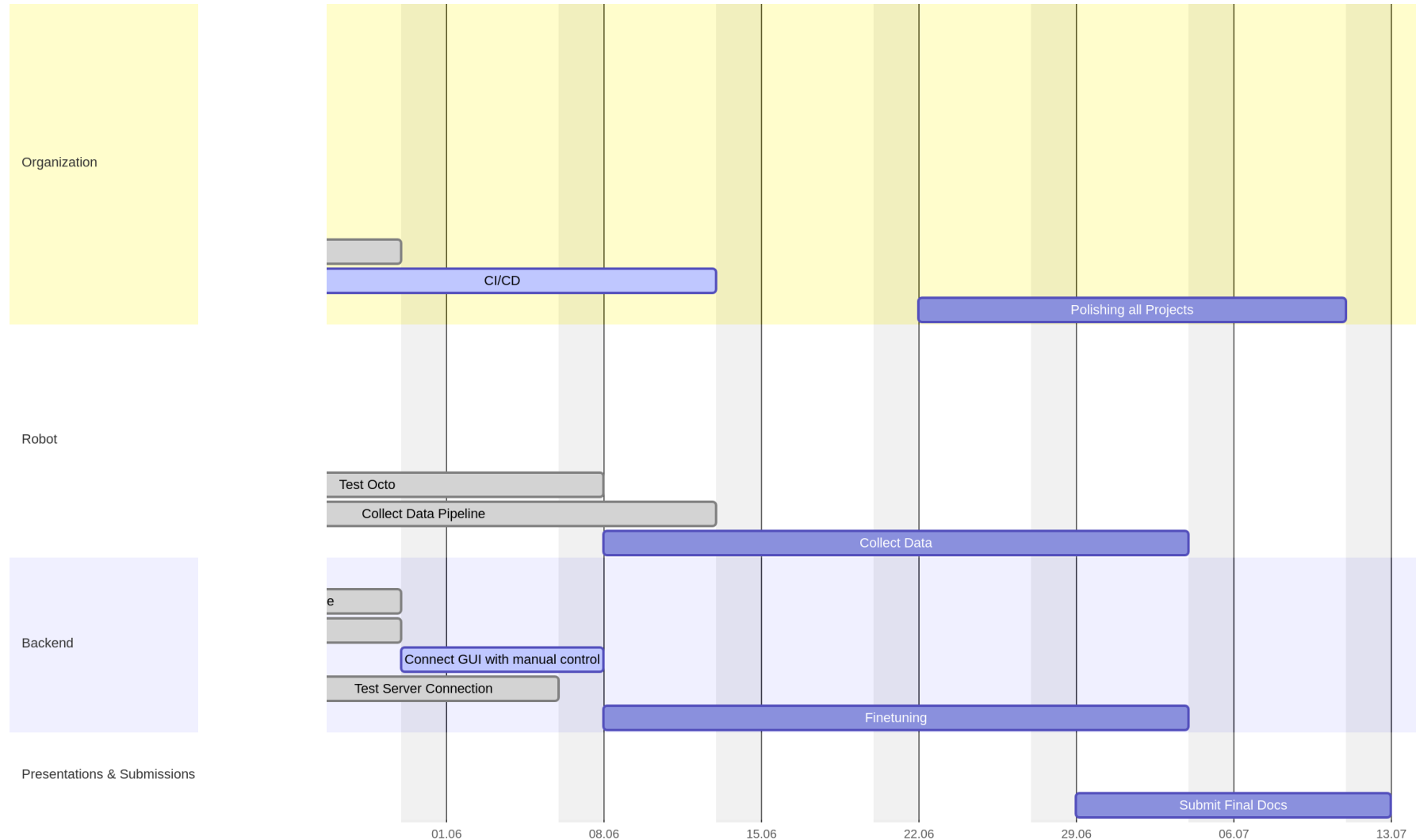


Brightened to handle dark lighting conditions

Challenges

- Synchronization problems between frontend and backend
- Zero-Calibration of motors
- Controller drift
- GPU mismatch

◆ GANTT Chart (Planned for Milestone 3)



◆ Reference List

- [1] CI/CD Pipeline: <https://wandb.ai/site/de/articles/ci-cd-mlops/?e-filter-32fa548-course-category=wb-platform>
- [2] ISIS Kurs Entwurf Eingebetteter Systeme: <https://isis.tu-berlin.de/course/view.php?id=48172>
- [3] Pi Build Hat: <https://assets.raspberrypi.com/static/87019ddbbec1433704d57909cbafdd3b/6e7df/build-hat-hero.png>
- [4] a16z (GPU Cluster): <https://a16z.com>
- [5] LEGO Force Sensor: <https://i.ebayimg.com/images/g/yzUAAOSwqUZhD4Tm/s-l1200.jpg>
- [6] Source Robotics (Witness Marks): <https://source-robotics.com>
- [7] Octo: <https://arxiv.org/pdf/2405.12213>
- [8] Time series prediction:
https://www.reddit.com/r/robotics/comments/1pms3ie/robotic_arm_controlled_by_vlmvision_language_model/?tl=de
- [9] X-Box Controller: <https://www.microsoft.com/de-de/d/xbox-wireless-controller-usb-c-kabel/8t8kcnb1xs3d>
- [10] Mathworks Kinematics: <https://www.mathworks.com/discovery/inverse-kinematics.html>
- [11] Open X-Embodiment: Robotic Learning Datasets and RT-X Models: <https://arxiv.org/pdf/2310.08864>
- [12] Visualizing Open X-Embodiment Dataset in Foxglove: <https://foxglove.dev/blog/visualizing-open-x-embodiment-dataset-in-foxglove>

Questions?

